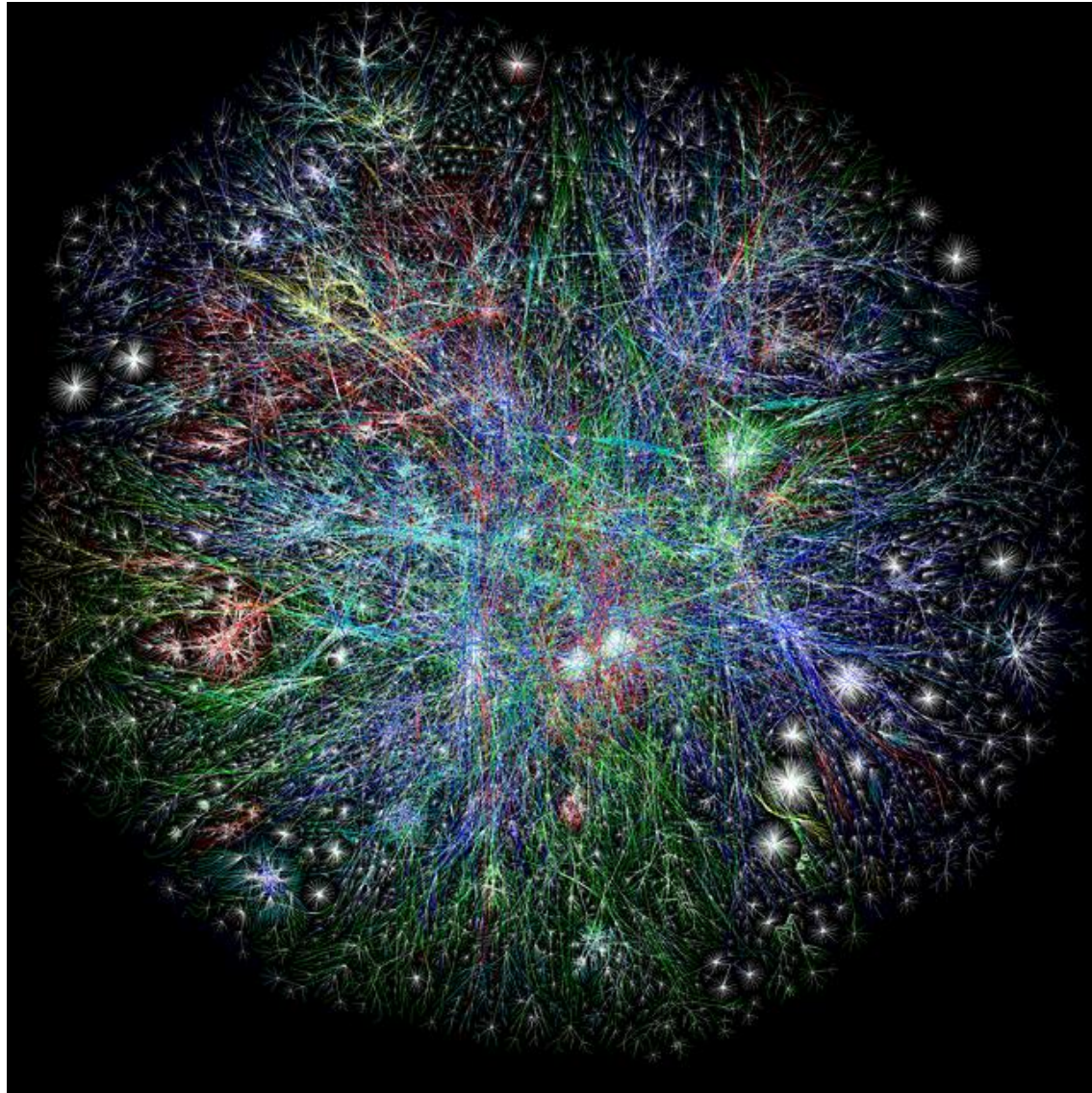


COMMUNITY DETECTION

Graph Data!

Internet



Graph Data: Social Networks

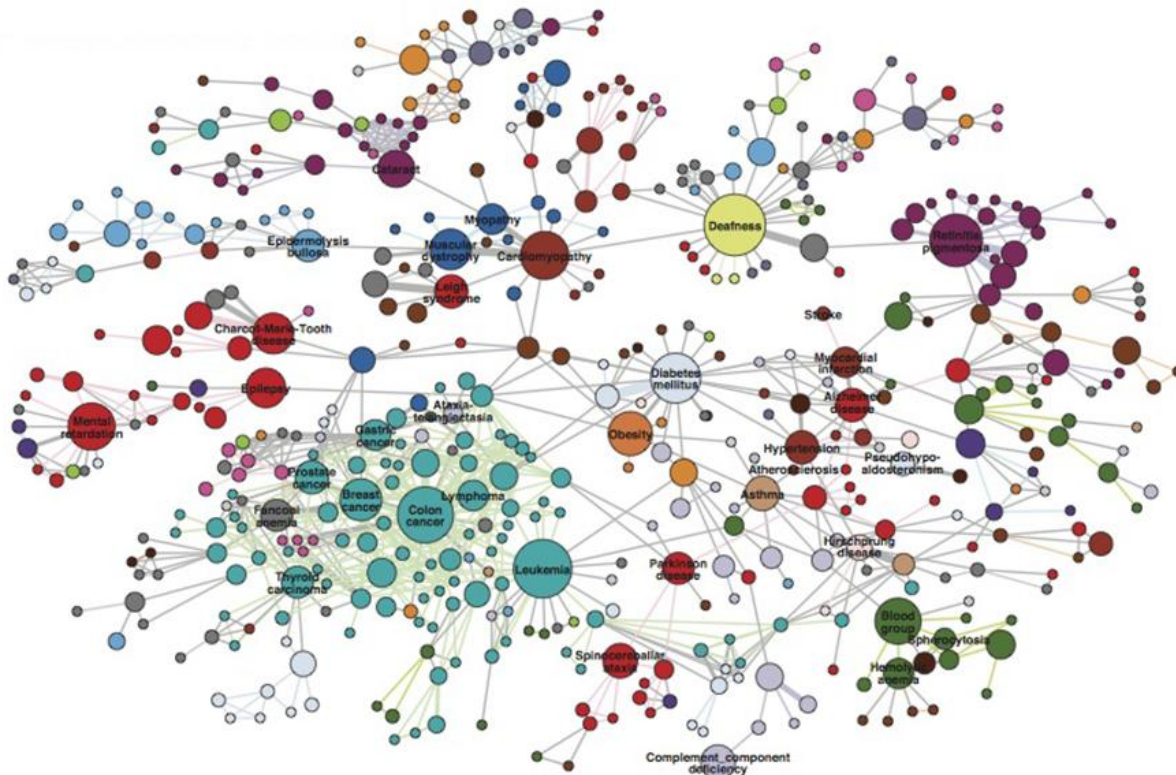


Facebook social graph

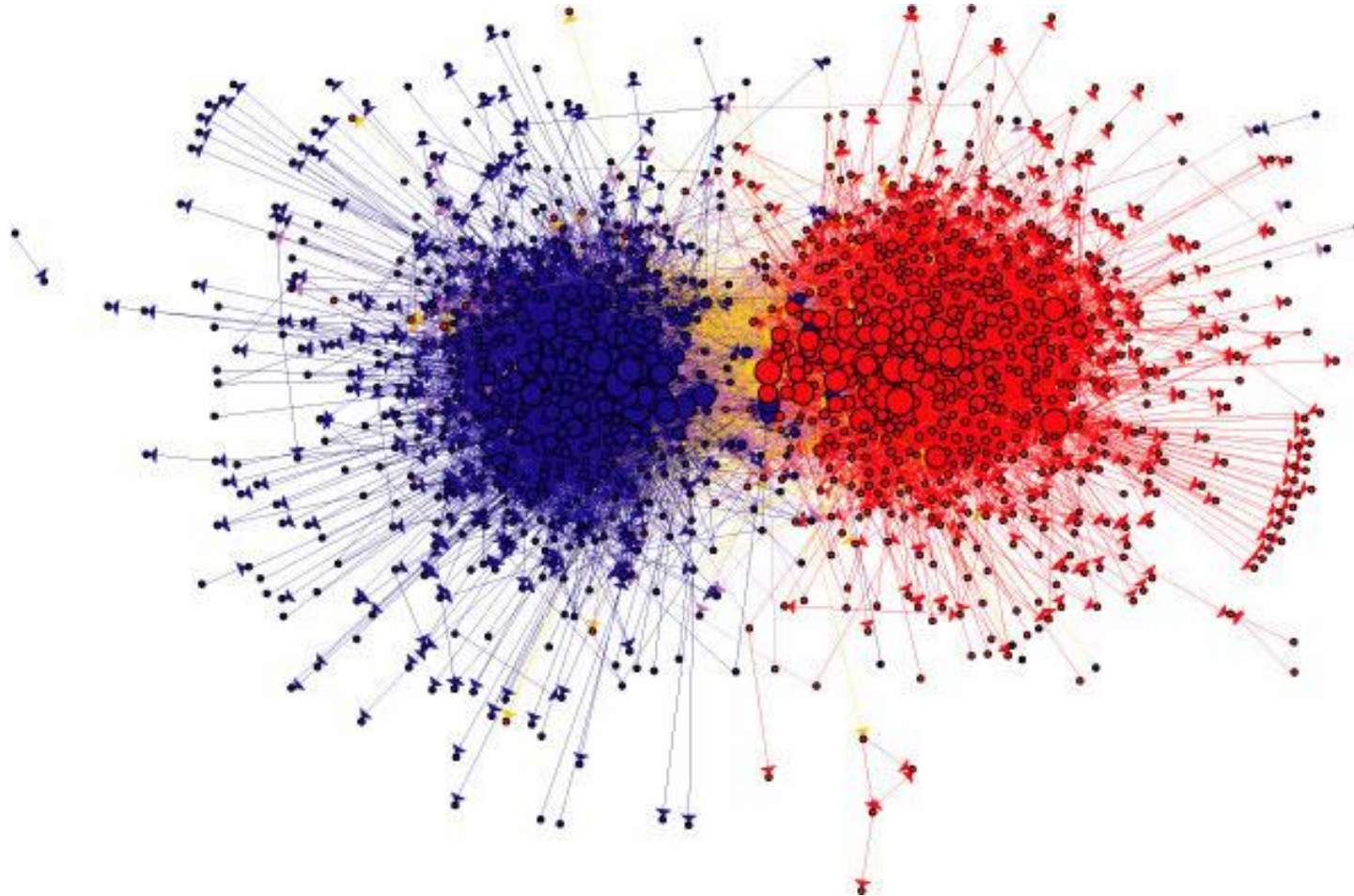
4-degrees of separation [Backstrom-Boldi-Rosa-Ugander-Vigna, 2011]

Biological Networks

- Genetic disorders and gene associations
- Drugs and their protein targets
- ...

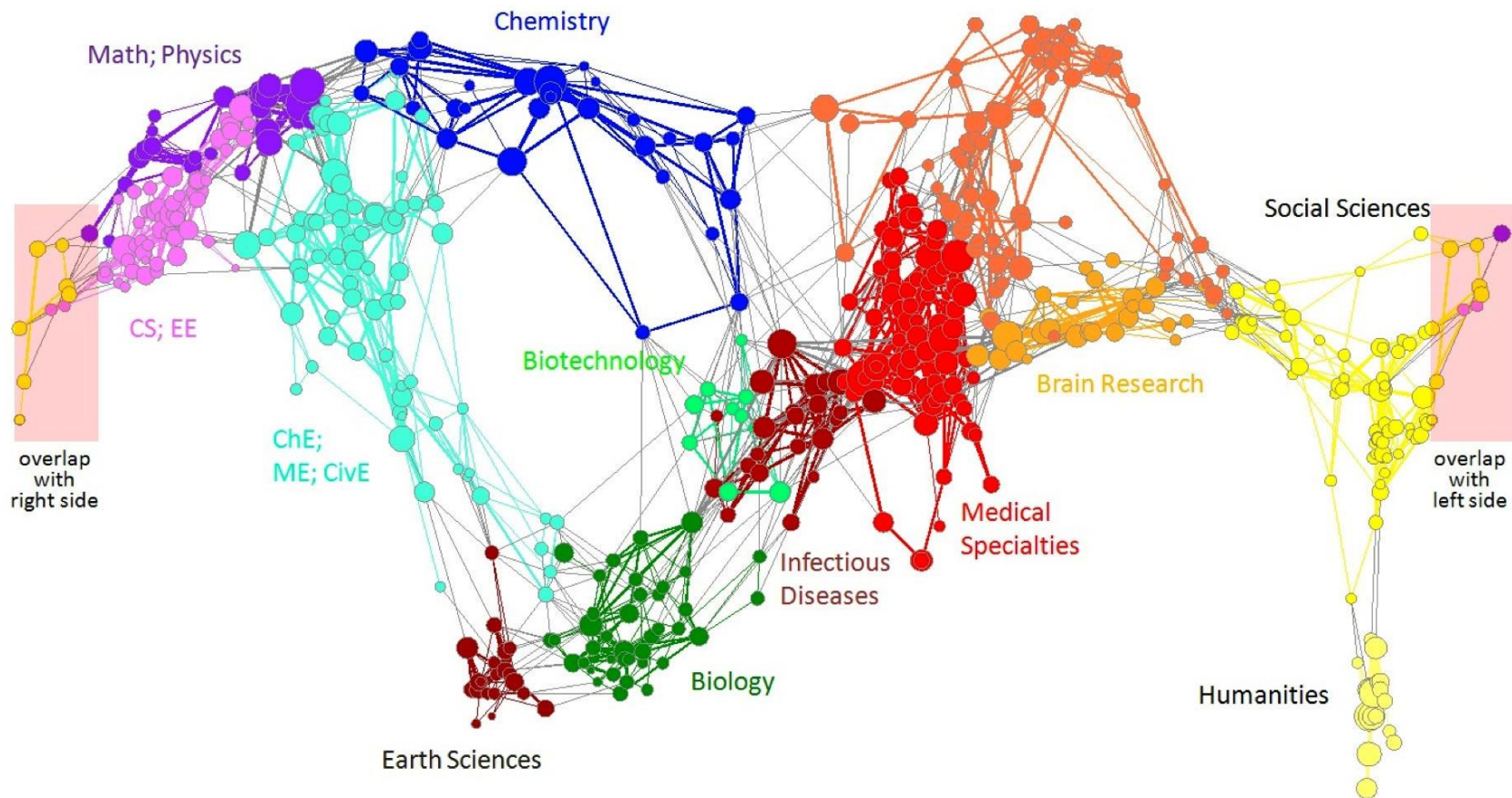


Media Networks



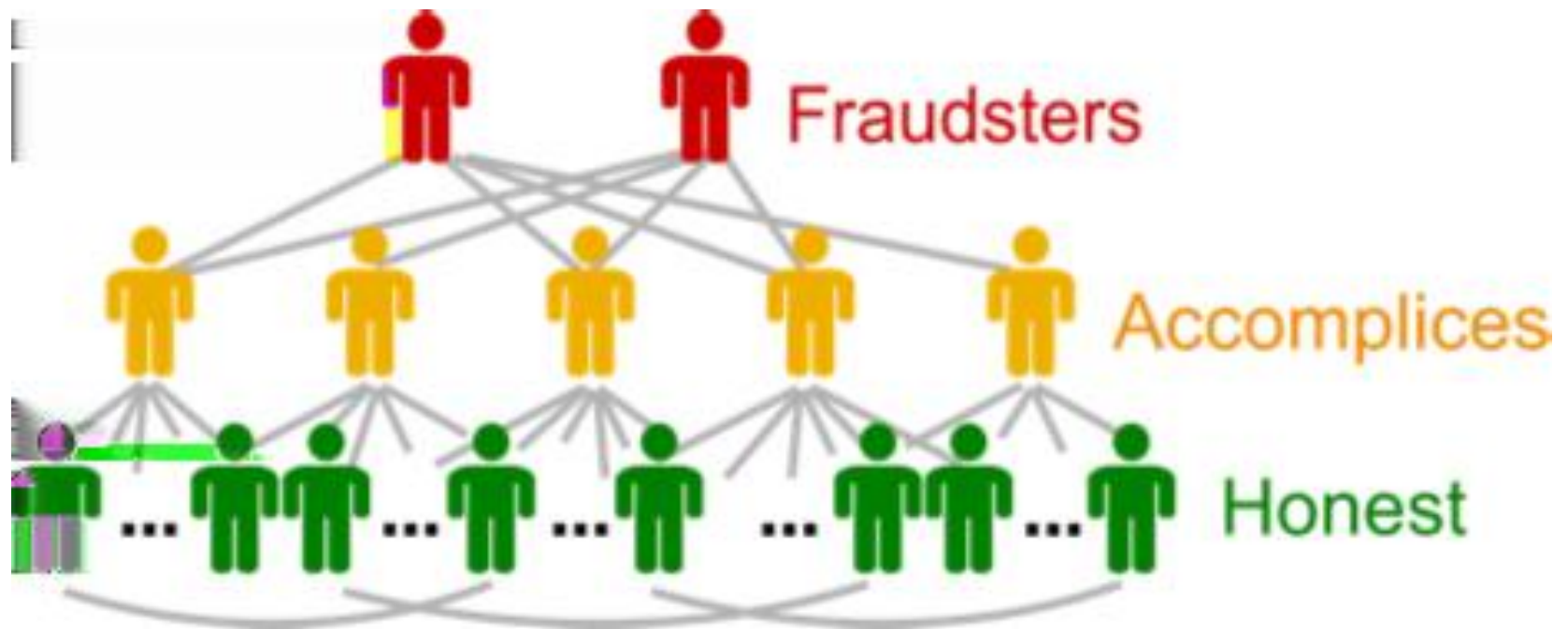
Connections between political blogs
Polarization of the network [Adamic-Glance, 2005]

Graph Data: Information Nets



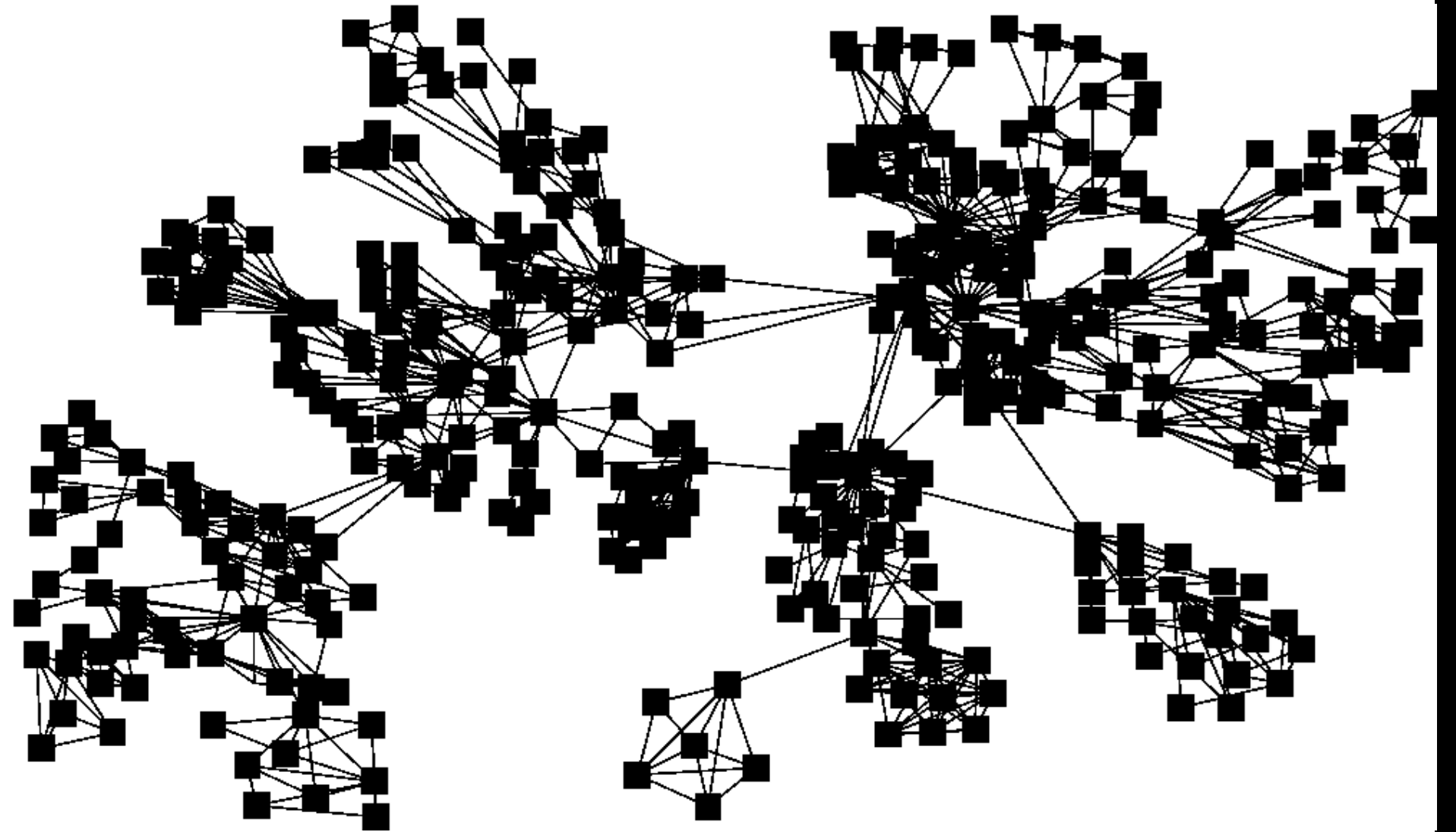
Citation networks and Maps of science
[Börner et al., 2012]

eBay Fraud Detection

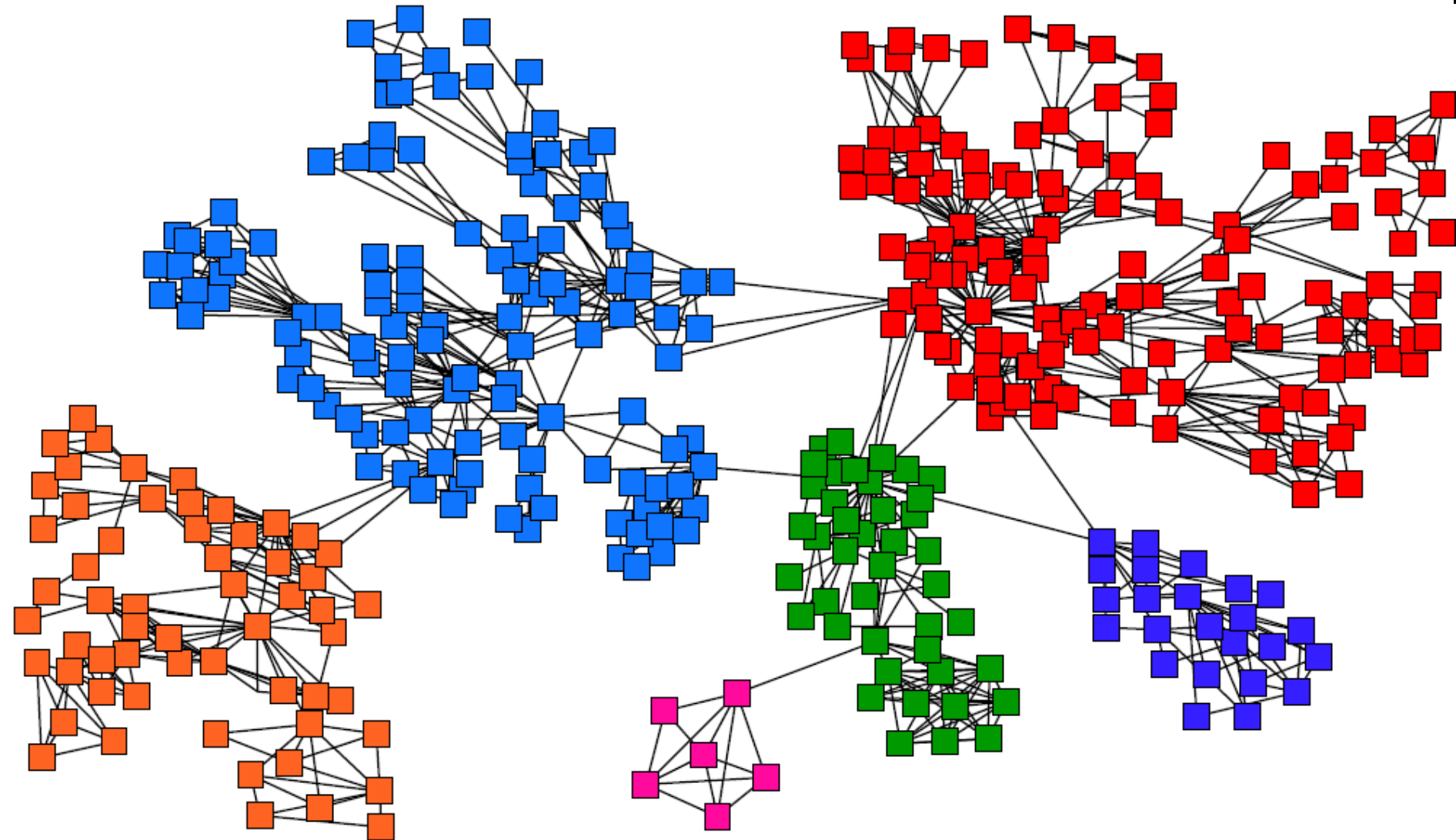


Communities

Goal: Find Densely Linked Clusters

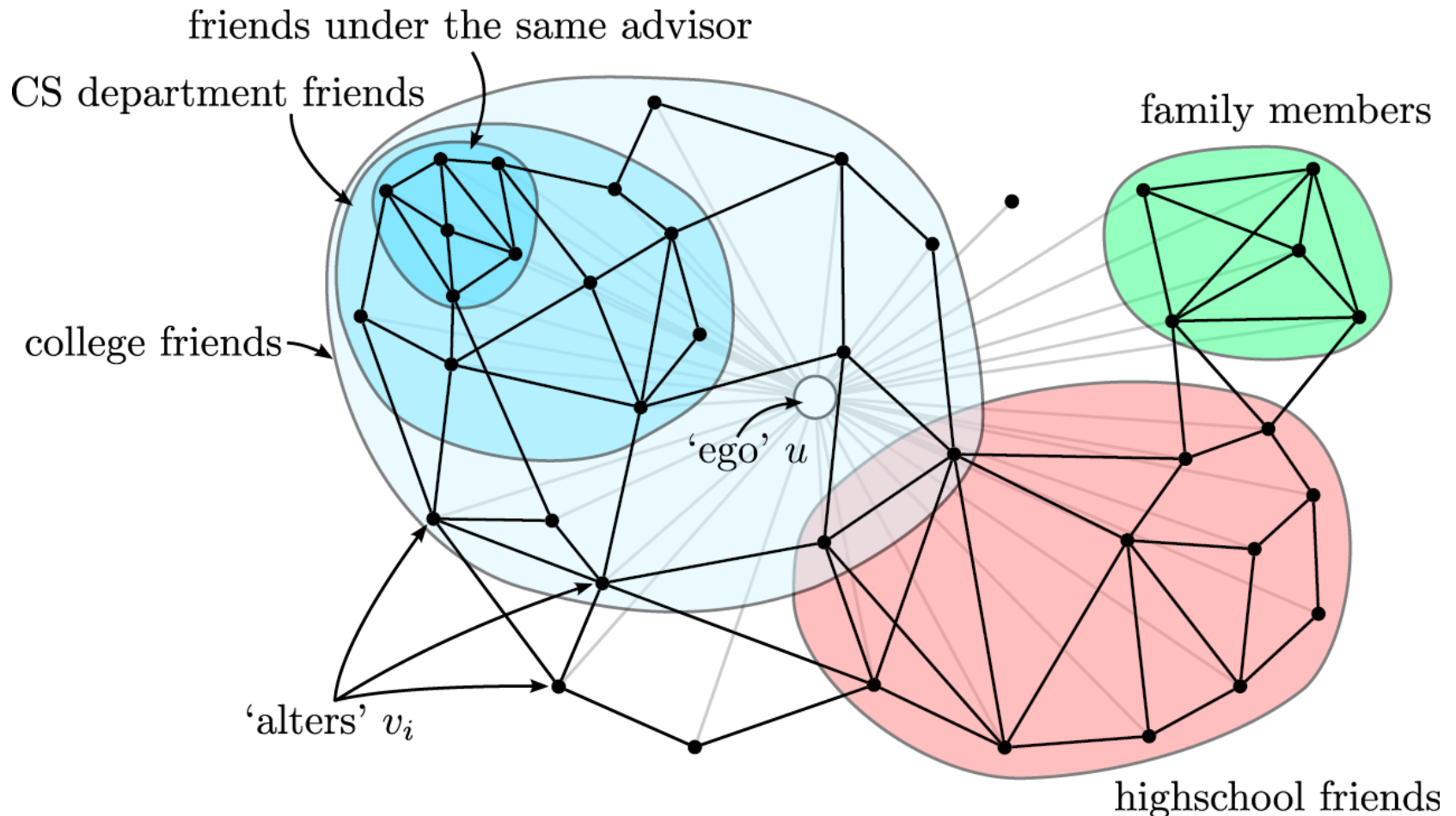


Goal: Find Densely Linked Clusters



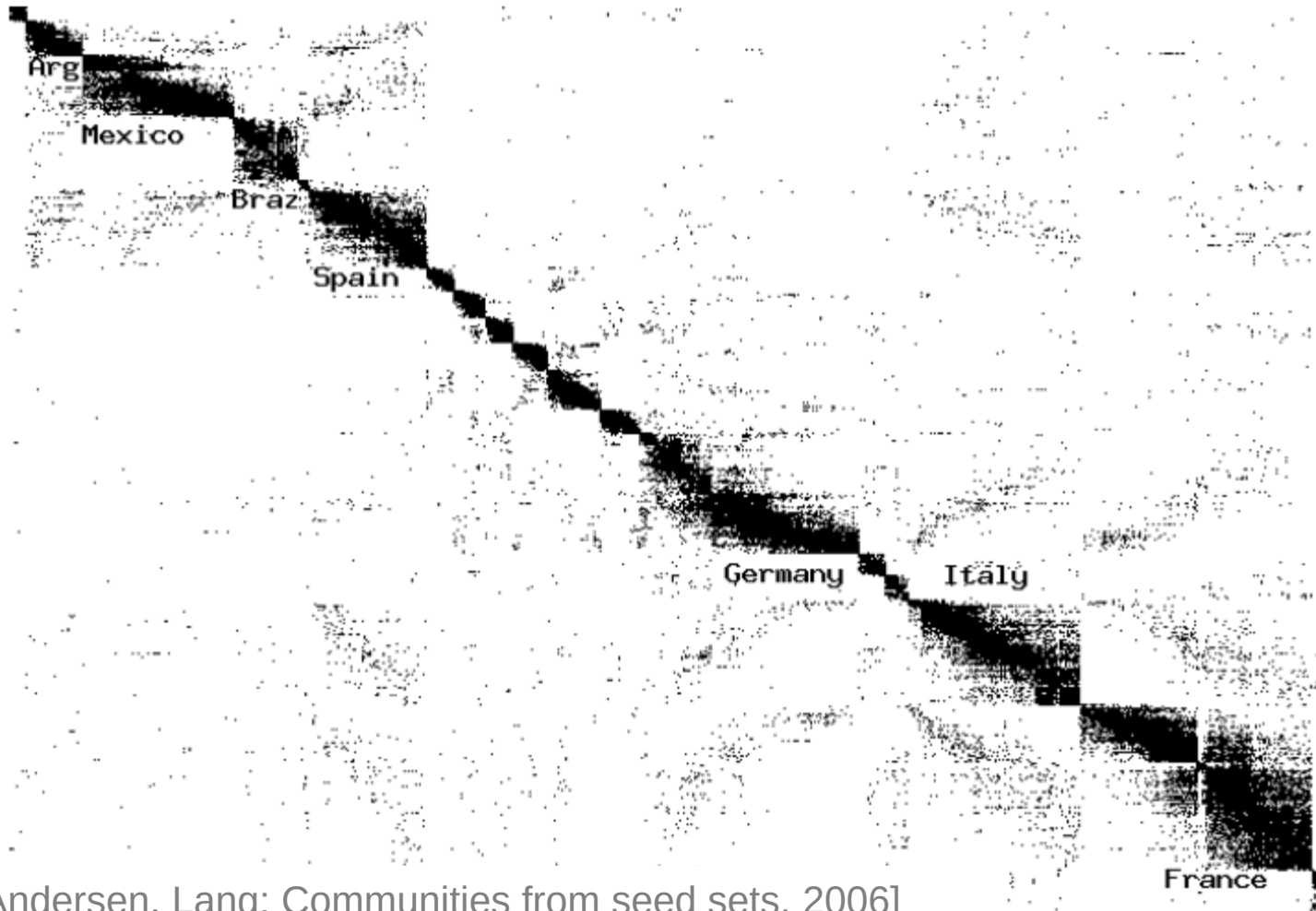
Twitter & Facebook

- Discovering **social circles**, circles of trust:



Movies and Actors

- Clusters in **Movies-to-Actors** graph:



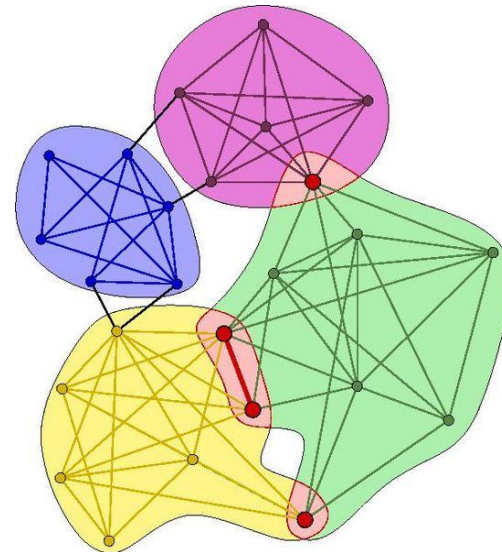
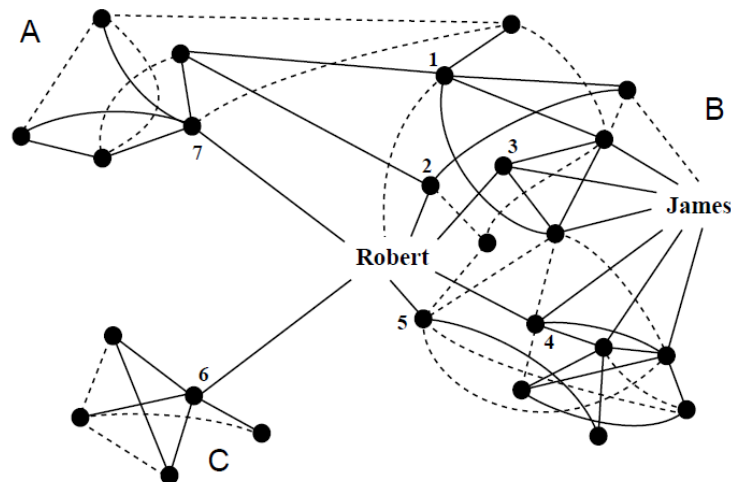
[Andersen, Lang: Communities from seed sets, 2006]

Community Detection

(undirected, unweighted networks)

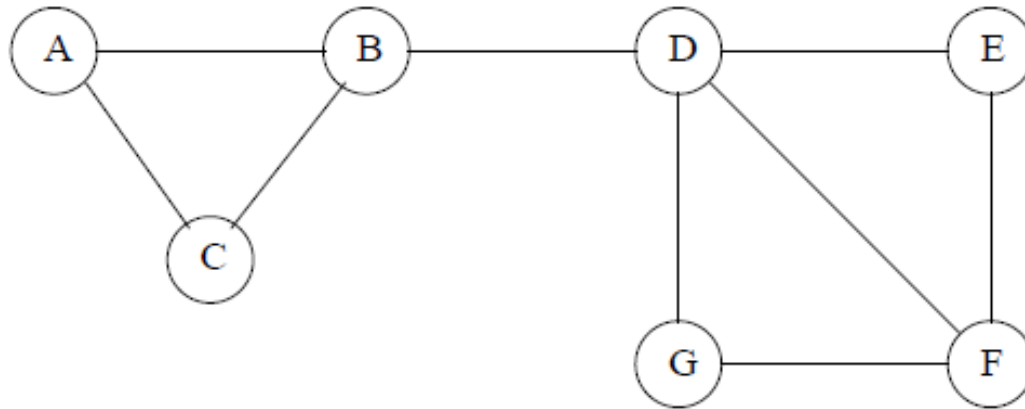
Problem Definition

- Given an undirected, unweighted graph
- Find **communities**:
 - Subgraphs that are dense
 - Not many edges between communities



Why Different than Clustering?

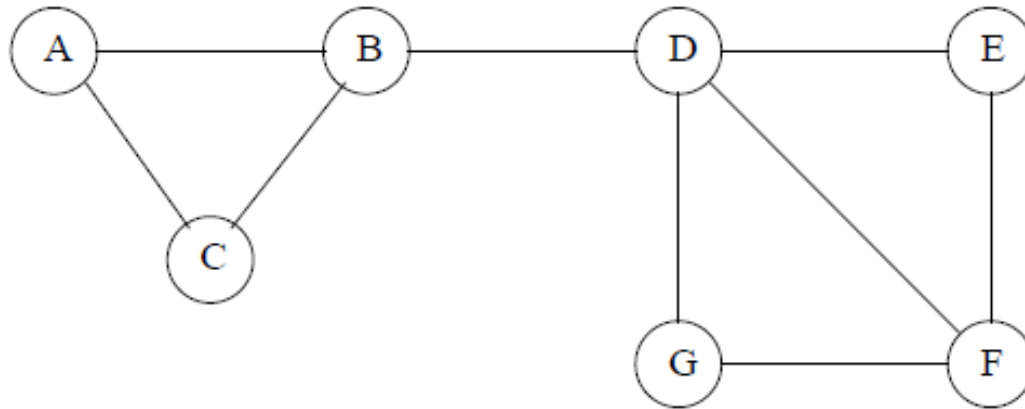
- Distance = graph distance



- Agglomerative clustering
 - Edges are all same distance
 - Pick any edge first

Why Different than Clustering?

- Distance = graph distance



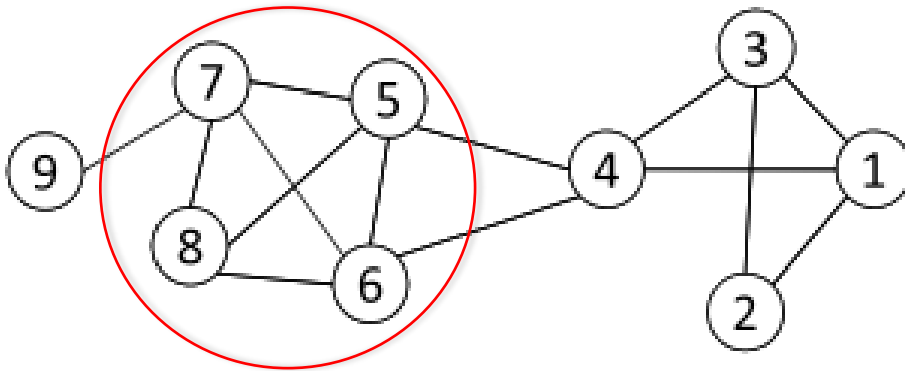
- Point assignment
 - Starting nodes: B and F
 - Where does D go?

How Would You Find Communities in a Graph?

- Subgraphs that are dense
- Not many edges between communities

Method 1: Cliques!

- **Clique**: a maximum complete subgraph in which all nodes are adjacent to each other



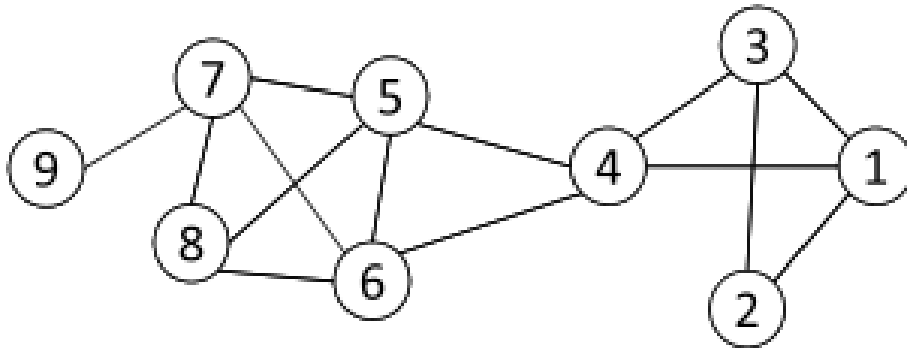
Nodes 5, 6, 7 and 8 form a clique

- **NP-hard** to find the maximum clique in a network
- Straightforward implementation to find cliques is very expensive in time complexity

Clique Percolation

- Clique is a very strict definition, unstable
- Normally use cliques as a core or a seed to find larger communities
 - **Input:** A parameter k , and a network
 - **Procedure**
 - Find out all cliques of size k in a given network
 - Construct a clique graph. Two cliques are adjacent if they share $k-1$ nodes
 - Each connected components in the clique graph form a community
- Clique percolation finds overlapping communities

Clique Percolation Example

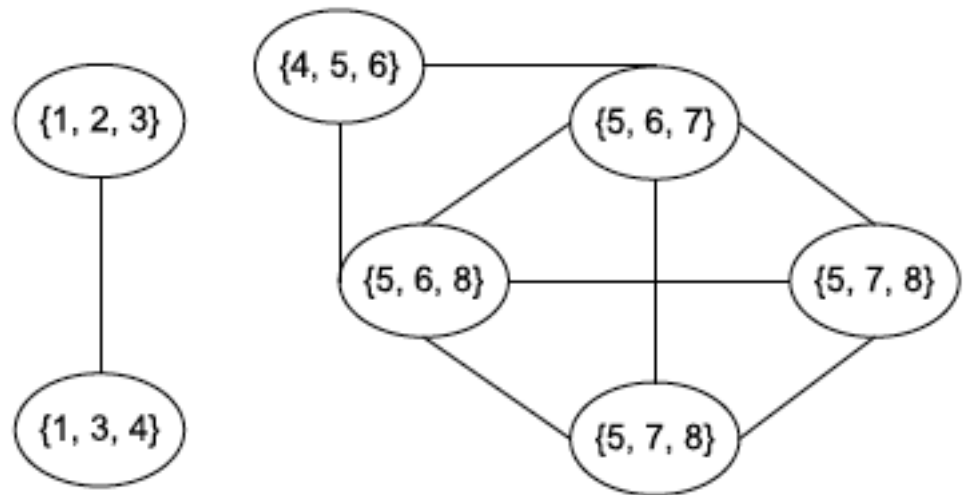


Cliques of size 3:

$\{1, 2, 3\}$, $\{1, 3, 4\}$, $\{4, 5, 6\}$, $\{5, 6, 7\}$, $\{5, 6, 8\}$, $\{5, 7, 8\}$, $\{6, 7, 8\}$

Communities:

$\{1, 2, 3, 4\}$
 $\{4, 5, 6, 7, 8\}$



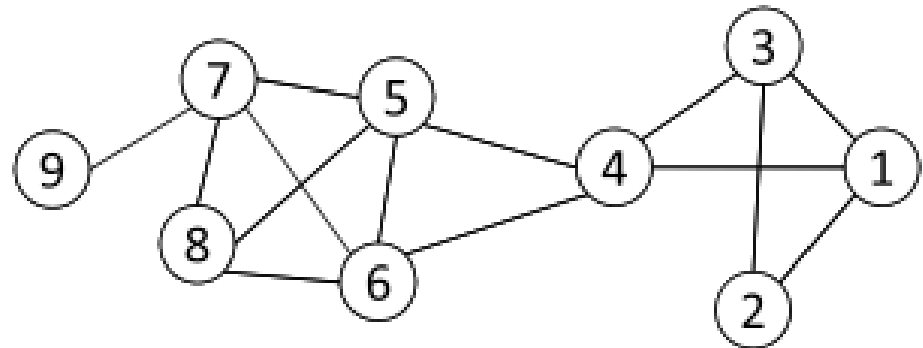
Problem with Clique Percolation

- Still expensive
- Assumes overlaps are not dense

Method 2: Clustering based on Vertex Similarity

- Apply k-means or similarity-based clustering to nodes
- Vertex similarity is defined in terms of **the similarity of their neighborhood**
- **Structural equivalence**: two nodes are structurally equivalent iff they are connecting to the same set of actors

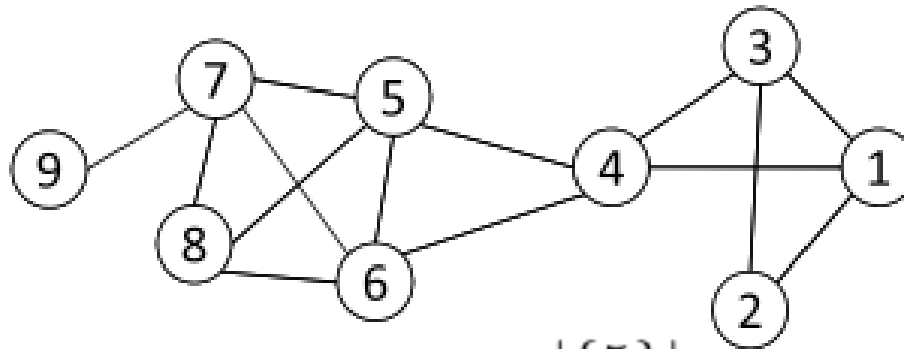
Nodes 1 and 3 are
structurally equivalent;
So are nodes 5 and 6.



- Structural equivalence is too strict for practical use.

Vertex Similarity

- Jaccard Similarity $Jaccard(v_i, v_j) = \frac{|N_i \cap N_j|}{|N_i \cup N_j|}$
- Cosine similarity $Cosine(v_i, v_j) = \frac{|N_i \cap N_j|}{\sqrt{|N_i| \cdot |N_j|}}$



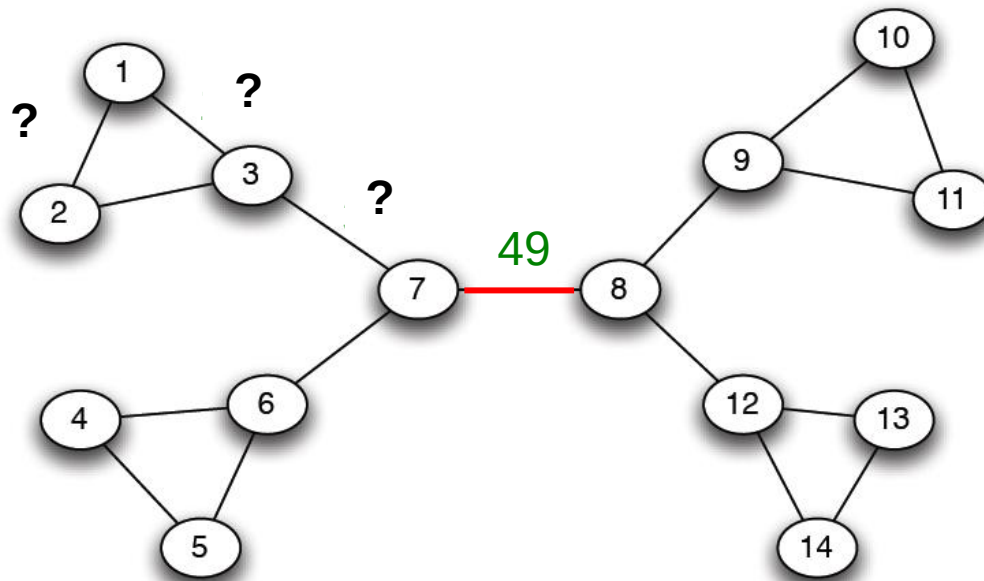
$$Jaccard(4, 6) = \frac{|\{5\}|}{|\{1, 3, 4, 5, 6, 7, 8\}|} = \frac{1}{7}$$

$$cosine(4, 6) = \frac{1}{\sqrt{4 \cdot 4}} = \frac{1}{4}$$

Method 3: Girvan-Newman

- Divisive hierarchical clustering based on the notion of edge **betweenness**:
 - *Number* of shortest paths passing through the edge
 - *Fraction* of shortest paths passing through the edge
 - Why interesting?

GIRVAN-NEWMAN: EXAMPLE

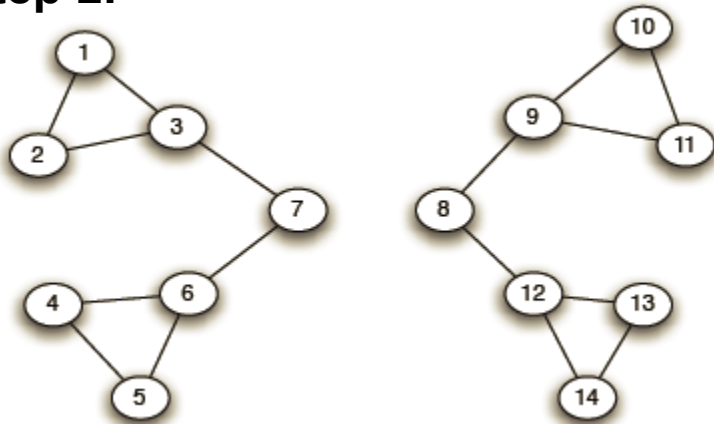


Method 3: Girvan-Newman

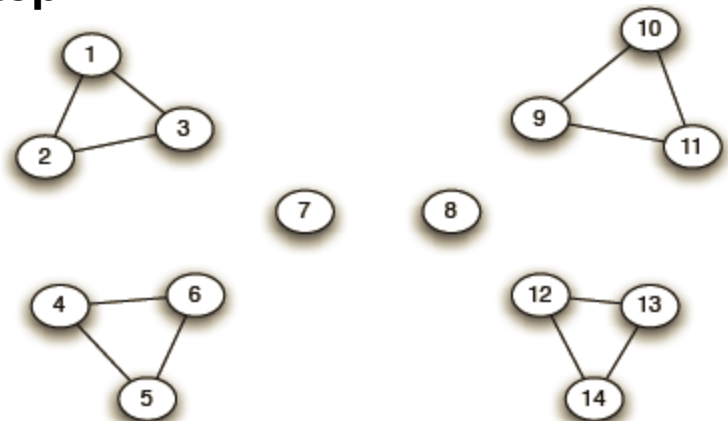
- Divisive hierarchical clustering based on the notion of edge **betweenness**:
 - *Number* of shortest paths passing through the edge
 - *Fraction* of shortest paths passing through the edge
- **Girvan-Newman Algorithm**:
 - **Repeat until no edges are left**:
 - Calculate betweenness of edges
 - Remove edges with highest betweenness
 - Connected components are communities
 - Gives a hierarchical decomposition of the network

GIRVAN-NEWMAN: EXAMPLE

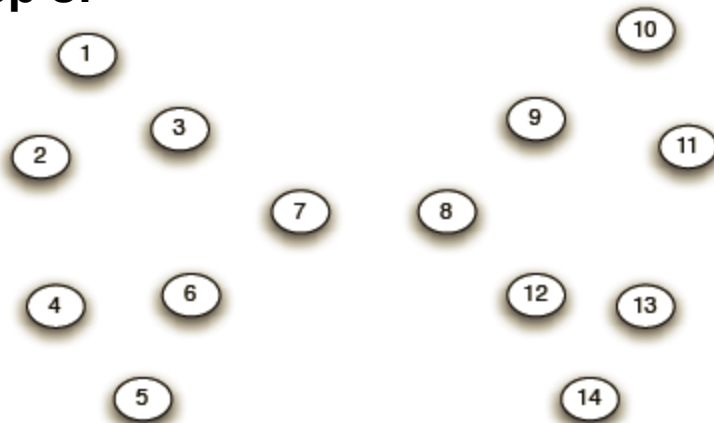
Step 1:



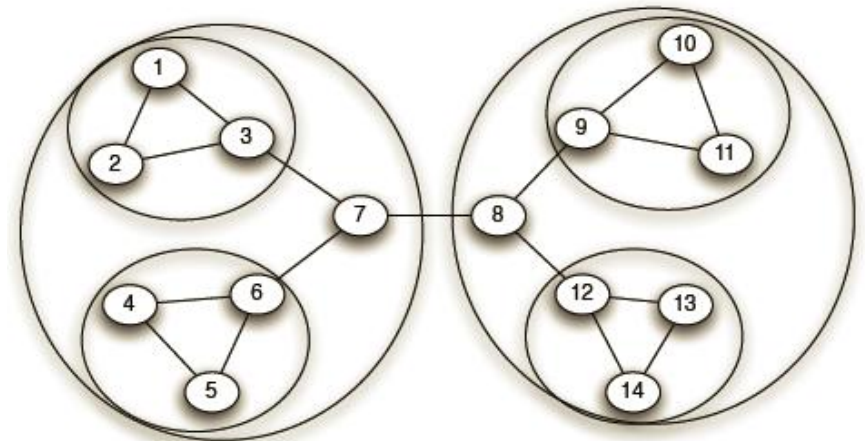
Step 2:



Step 3:

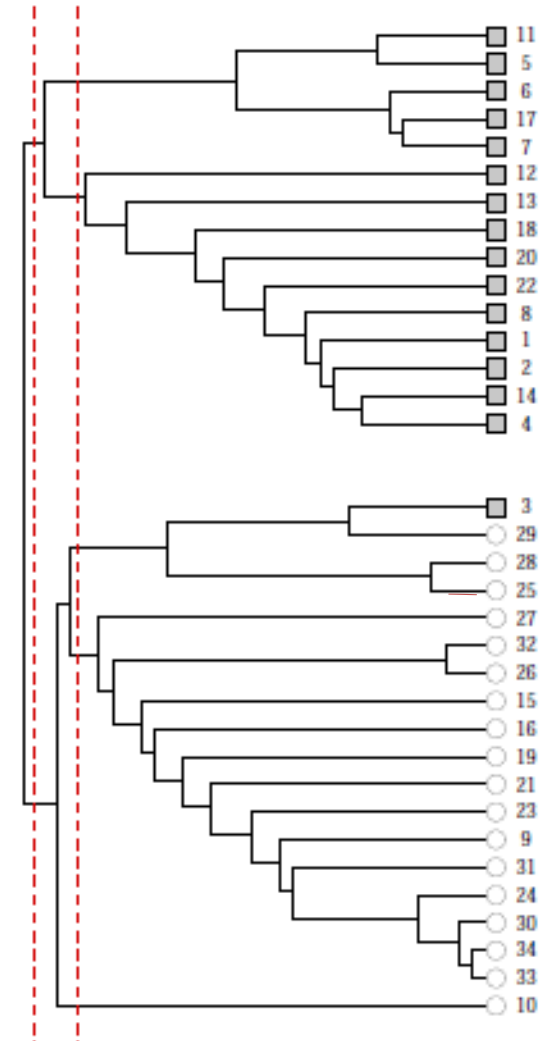
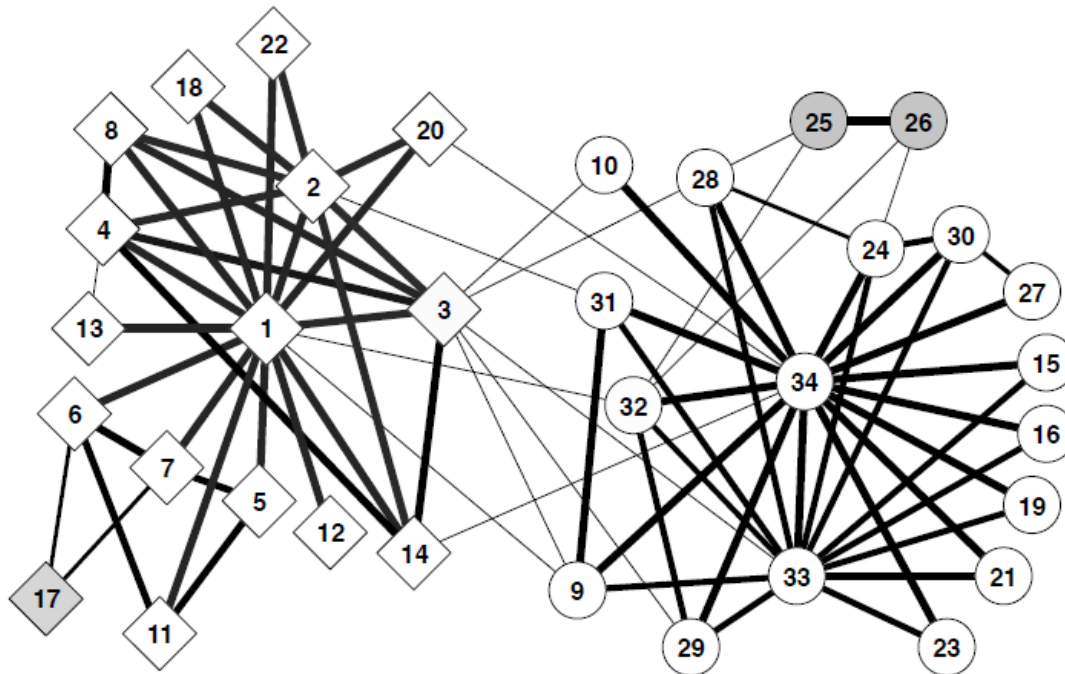


Hierarchical network decomposition:

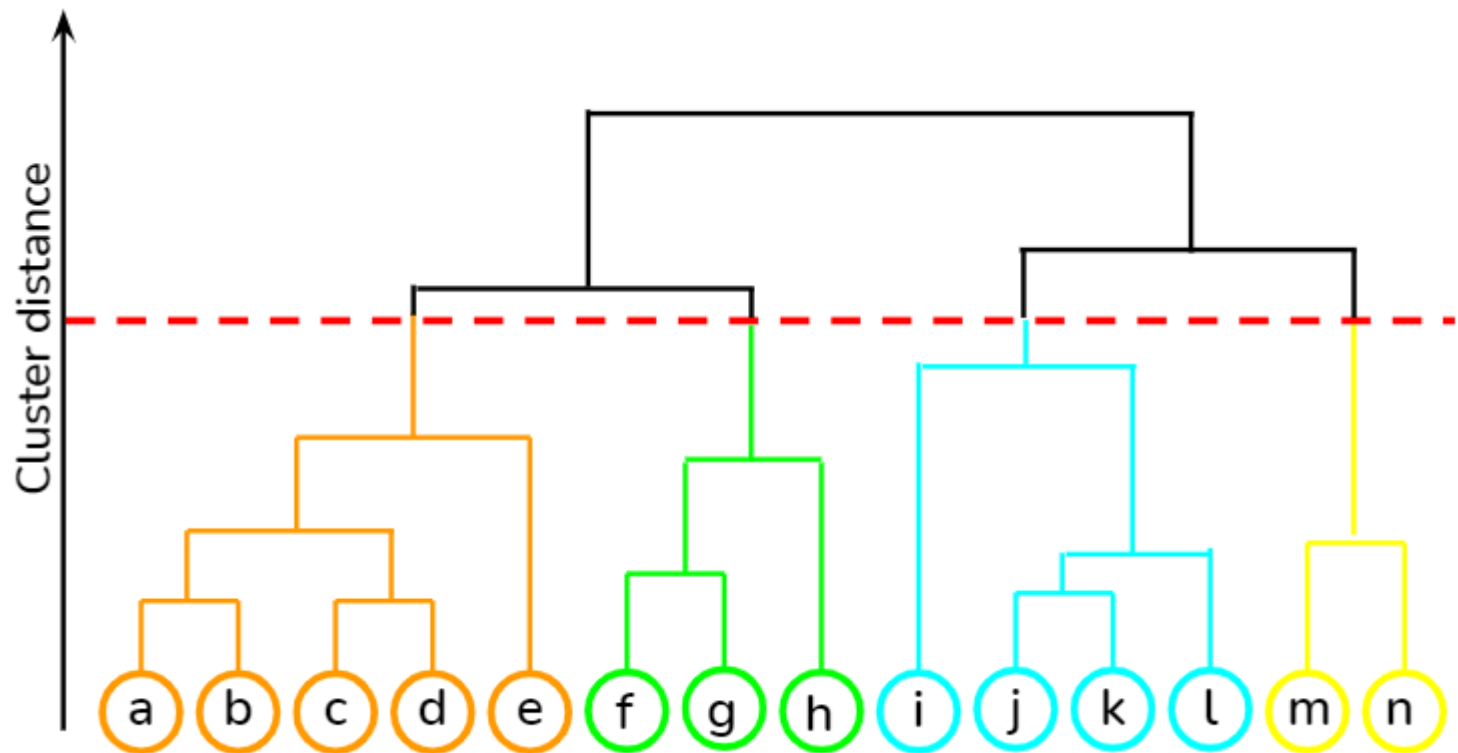


Girvan-Newman: Results

- Zachary's Karate club:
Hierarchical decomposition



Dendrogram



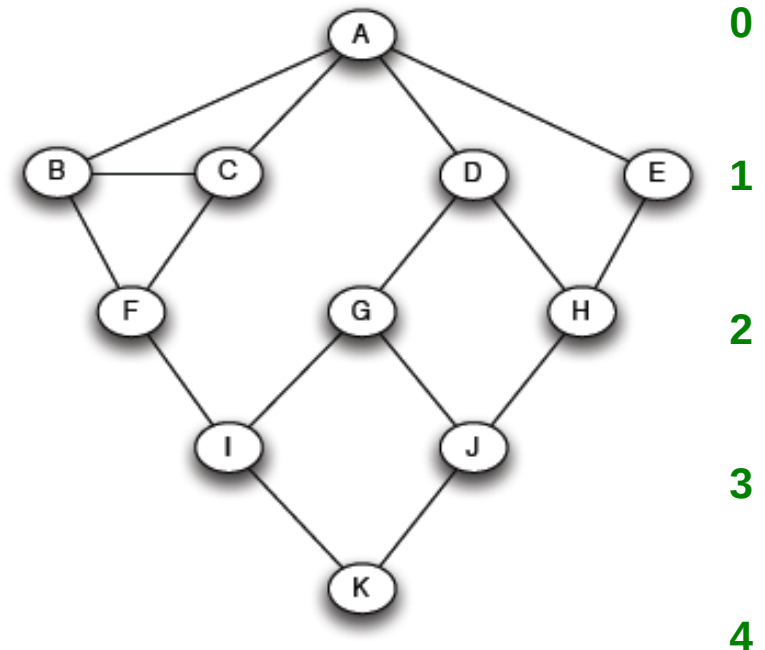
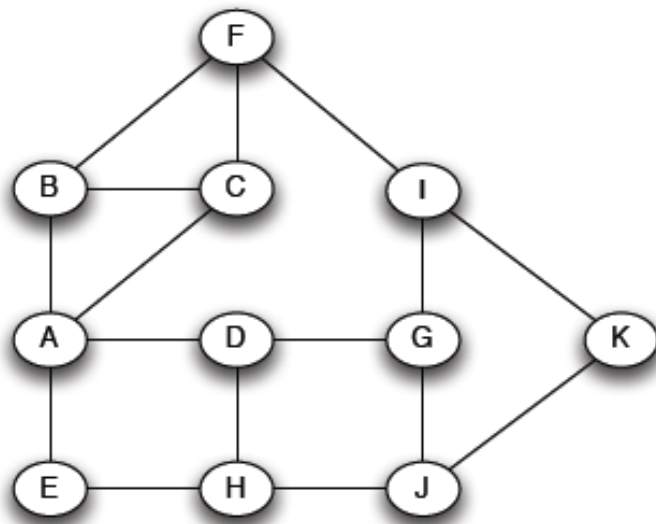
Two Questions Left

- How to compute betweenness?
- How to select the number of clusters?

How to Compute Betweenness?

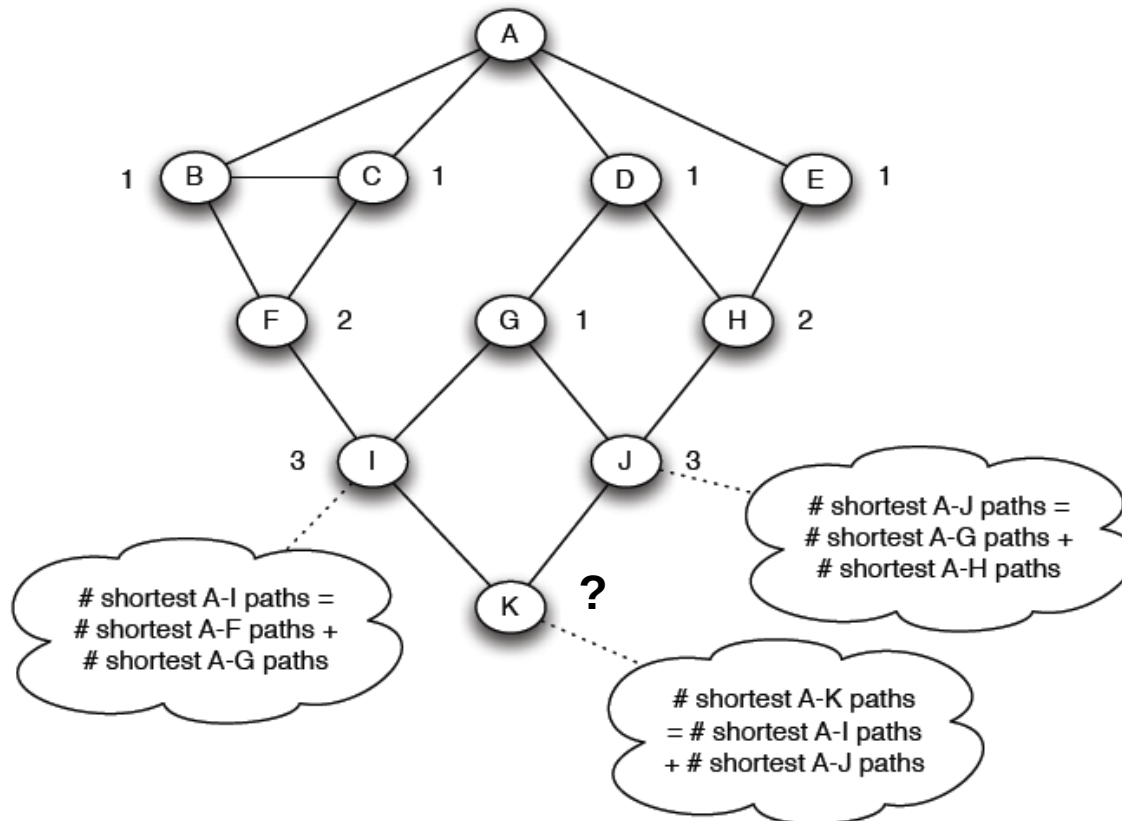
- Want to compute betweenness of paths starting at node *A*

BFS starting from *A*:



How to Compute Betweenness?

- Count the number of shortest paths from *A* to all other nodes of the network:



How to Compute Betweenness

- Compute **fractional** betweenness by working up the tree: If there are multiple paths count them fractionally

The algorithm:

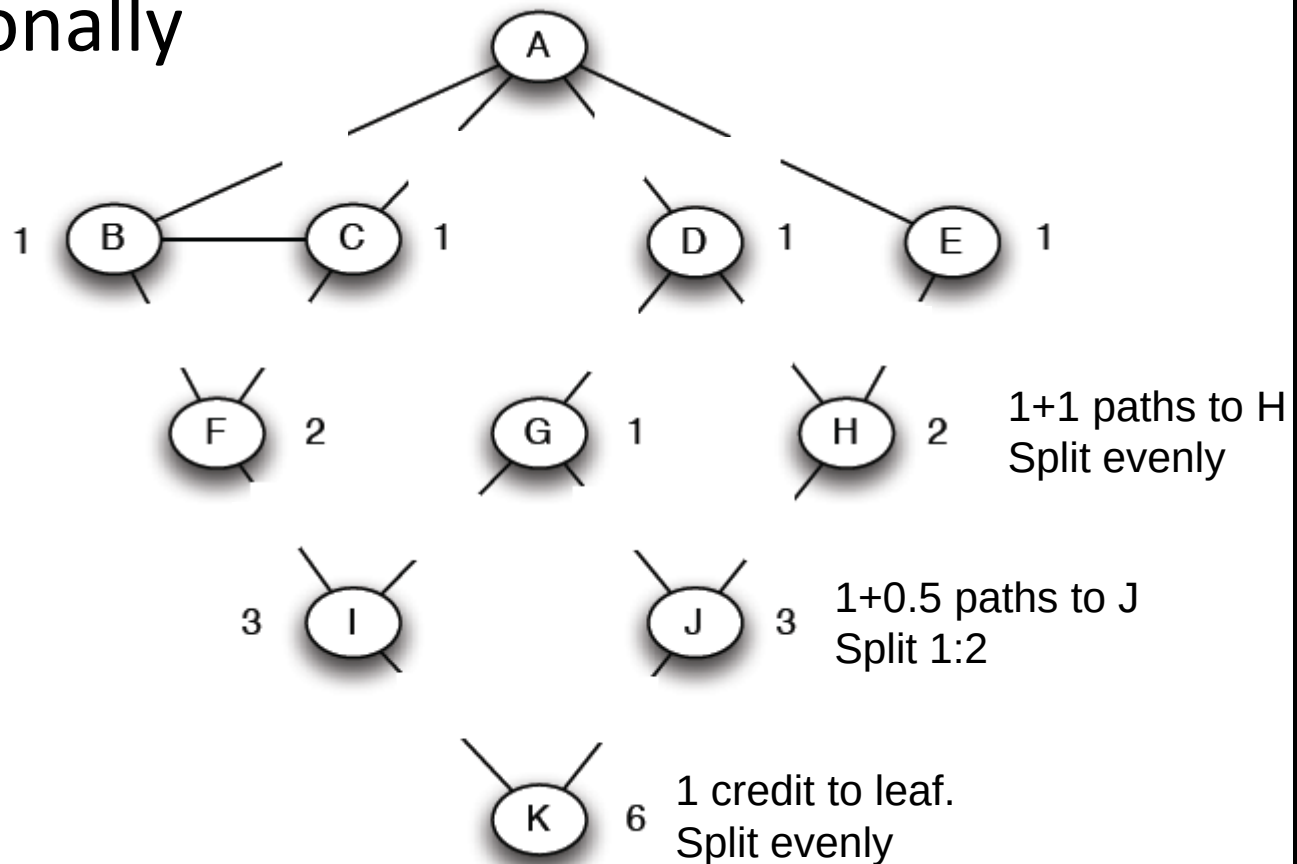
• Add edge flows:

-- node flow =

$$1 + \sum \text{child edges}$$

-- split the flow up based on the parent value

- Repeat the BFS procedure for each starting node U



How to Compute Betweenness?

- Sum credits for each edge
- Divide by 2
 - Each shortest path discovered twice (why?)

Two Questions Left

- How to compute betweenness?
- How to select the number of clusters?

Network Communities

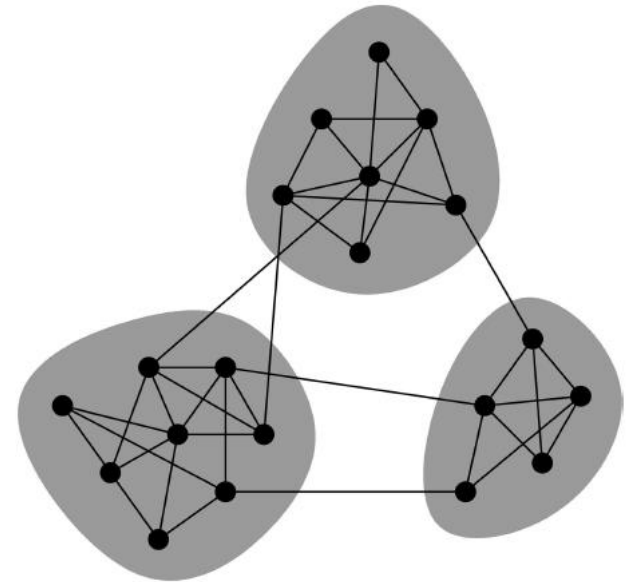
- **Communities:** sets of tightly connected nodes

- Define: **Modularity Q**

- A measure of how well a network is partitioned into communities
- Given a partitioning of the network into groups $s \in S$:

$$Q \propto \sum_{s \in S} [\underbrace{(\# \text{ edges within group } s) - (\text{expected } \# \text{ edges within group } s)}]$$

Need a null model!



Null Model: Configuration Model

- **Given real G on n nodes and m edges, construct rewired network G'**
 - Same degree distribution but random connections
 - Break each edge into two “stubs”
 - Consider G' as a **multigraph**

Null Model: Configuration Model

- The expected number of edges between nodes i and j of degrees k_i and k_j equals to: $k_i \cdot \frac{k_j}{2m}$

Note:

$$\#stubs = \sum_{u \in N} k_u = 2m$$

Modularity

- **Modularity of partitioning S of graph G :**

- $Q \propto \sum_{s \in S} [(\# \text{ edges within group } s) - (\text{expected } \# \text{ edges within group } s)]$

- $Q(G, S) = \frac{1}{2m} \sum_{s \in S} \sum_{i \in s} \sum_{j \in s} \left(A_{ij} - \frac{k_i k_j}{2m} \right)$

$A_{ij} = 1$ if $i \rightarrow j$,
0 else

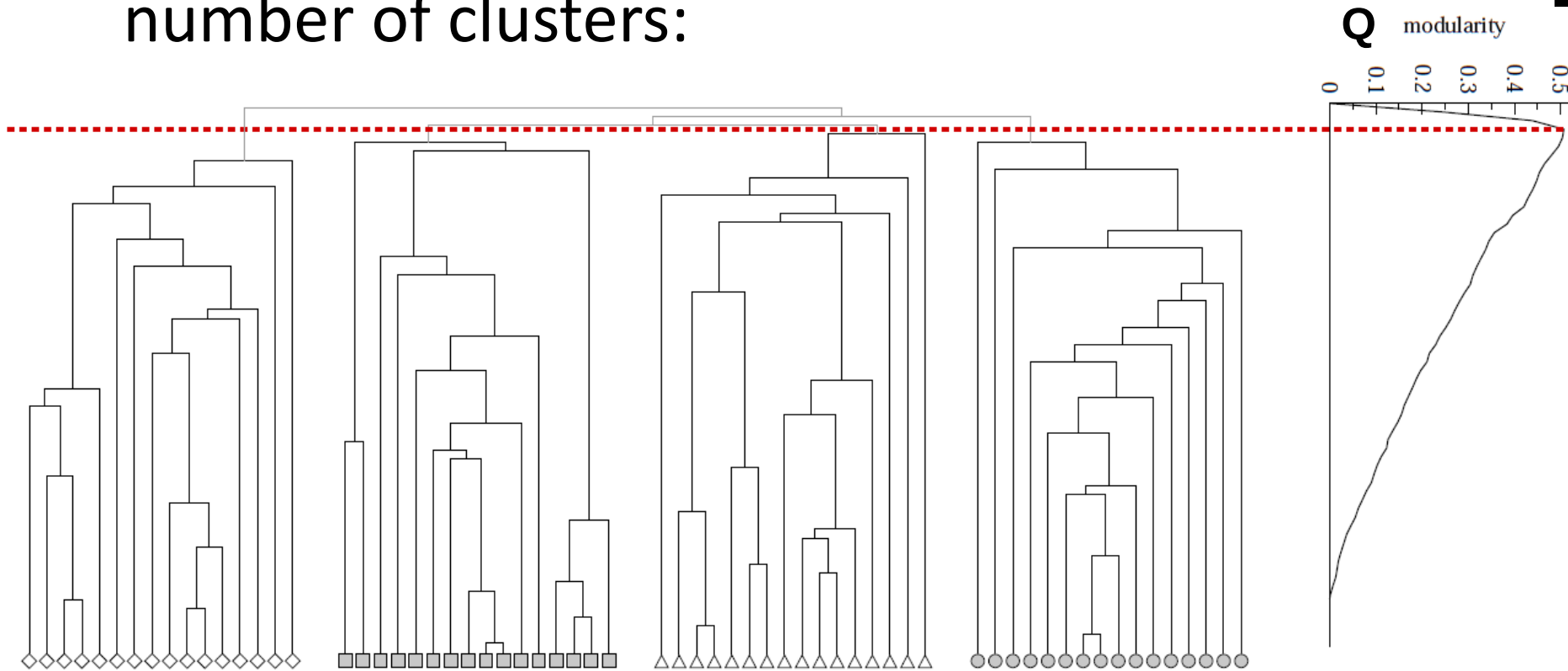
- **Modularity values take range $[-1,1]$**

- It is positive if the number of edges within groups exceeds the expected number

- **$-0.3 < Q < 0.7$** means significant community structure

Modularity: Number of clusters

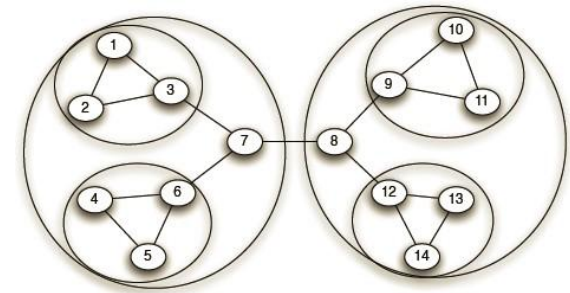
- Modularity is useful for selecting the number of clusters:



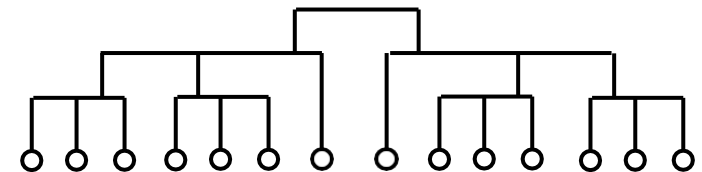
Louvain Algorithm

- **Greedy algorithm** for community detection
 - $O(n \log n)$ run time
- Supports weighted graphs
- Provides hierarchical communities
- Widely utilized to **study large networks** because:
 - Fast
 - Rapid convergence
 - High modularity output (i.e., “better communities”)

Network and communities:



Dendrogram:



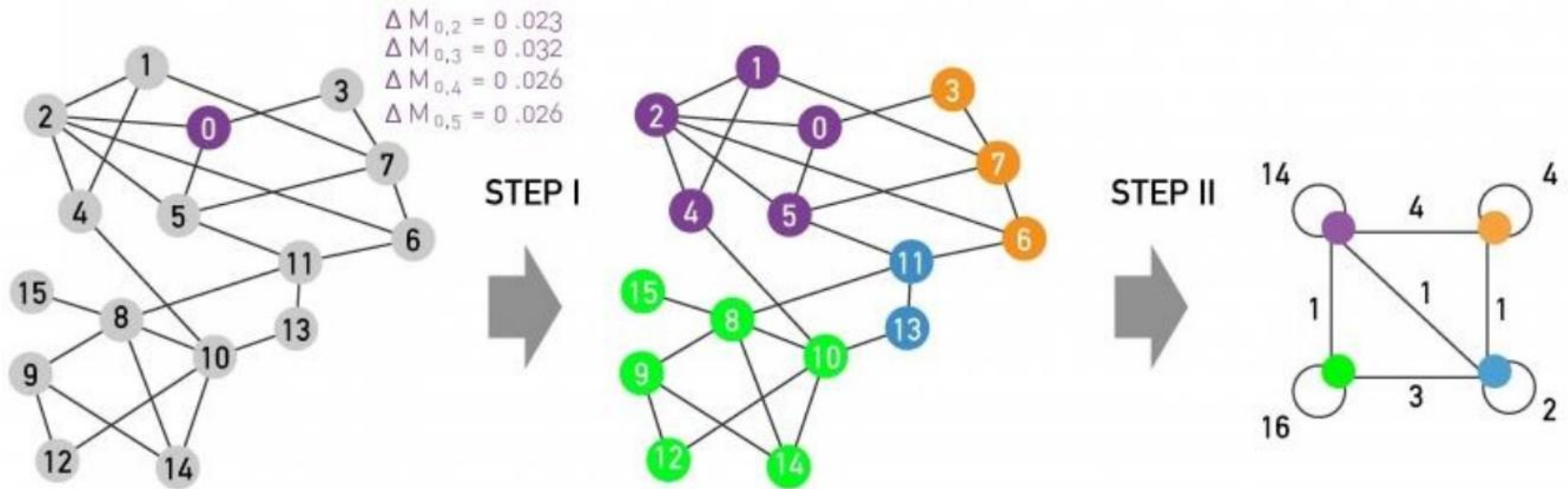
Louvain Algorithm: At High Level

- Louvain algorithm **greedily maximizes** modularity
- **Each pass is made of 2 phases:**
 - **Phase 1:** Modularity is **optimized** by allowing only local changes to node-communities memberships
 - **Phase 2:** The identified communities are **aggregated** into super-nodes to build a new network
 - **Goto Phase 1**

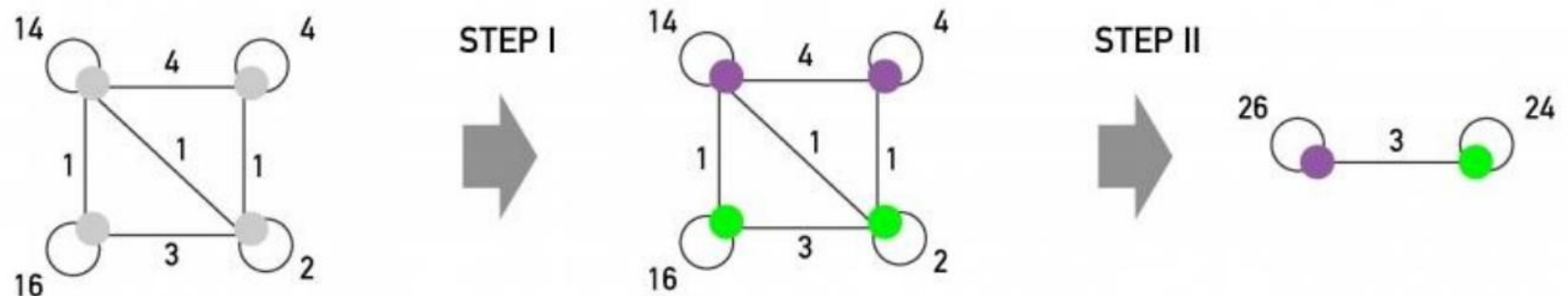
The passes are repeated **iteratively** until no increase of modularity is possible.

Louvain Algorithm

1ST PASS



2ND PASS



Louvain: 1st phase (Partitioning)

- Put each node in a graph into a **distinct community** (one node per community)
- For each node i , the algorithm performs two calculations:
 - Compute the modularity delta (ΔQ) when putting node i into the community of some neighbor j
 - Move i to a community of node j that yields the largest gain in ΔQ
- **Phase 1 runs until no movement yields a gain**

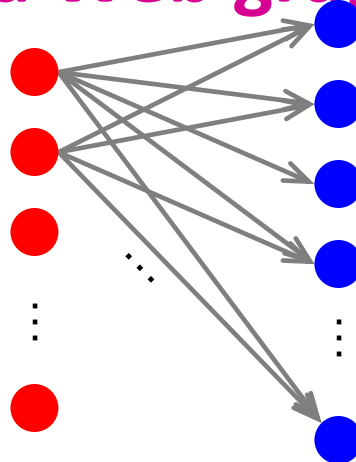
Louvain: 2nd phase (Restructuring)

- The communities obtained in the first phase are contracted into **super-nodes**, and the network is created accordingly:
 - Super-nodes are connected if there is at least one edge between the nodes of the corresponding communities
 - The weight of the edge between the two super-nodes is the sum of the weights from all edges between their corresponding communities
- **Phase 1 is then run on the super-node network**

Trawling

Trawling

- Searching for small communities in the Web graph
- What is the signature of a community / discussion in a Web graph?



Dense 2-layer graph

Use this to define “topics”:
What the same people on the left talk about on the right

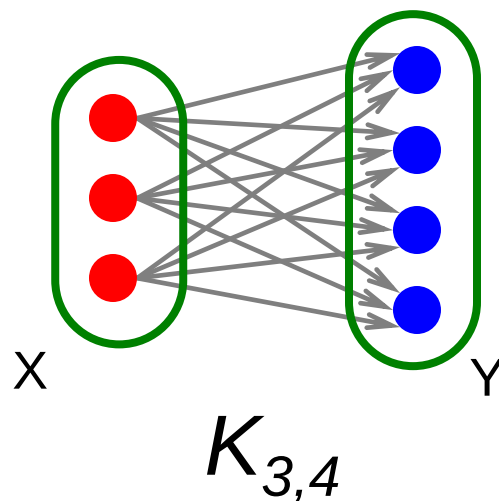
Intuition: Many people all talking about the same things

Searching for Small Communities

- **A more well-defined problem:**

Enumerate complete bipartite subgraphs $K_{s,t}$

–Where $K_{s,t}$: s nodes on the “left” where each links to the same t other nodes on the “right”



$$\begin{aligned} |X| &= s = 3 \\ |Y| &= t = 4 \end{aligned}$$

Fully connected

COMMUNITY EVALUATION

Evaluating Community Detection (1)

- For groups with clear definitions
 - E.g., Cliques, k-cliques, k-clubs, quasi-cliques
 - Verify whether extracted communities satisfy the definition
- For networks with ground truth information
 - Accuracy of pairwise community memberships

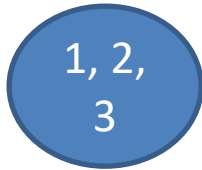
Accuracy of Pairwise Community Memberships

- Consider all the possible pairs of nodes and check whether they reside in the same community
- An **error** occurs *if*
 - Two nodes belonging to the **same** community are assigned to **different** communities
 - Two nodes belonging to **different** communities are assigned to the **same** community
- Construct a **contingency table or confusion matrix**

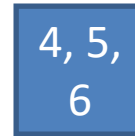
Clustering Result		Ground Truth	
		$C(v_i) = C(v_j)$	$C(v_i) \neq C(v_j)$
	$C(v_i) = C(v_j)$	a	b
	$C(v_i) \neq C(v_j)$	c	d

$$accuracy = \frac{a + d}{a + b + c + d} = \frac{a + d}{n(n - 1)/2}$$

Accuracy Example



Ground Truth



Algorithm Result

		Ground Truth	
		$C(v_i) = C(v_j)$	$C(v_i) \neq C(v_j)$
Algorithm Result	$C(v_i) = C(v_j)$		
	$C(v_i) \neq C(v_j)$		

Accuracy =

Evaluation using Semantics

- For networks with semantics
 - Networks come with semantic or attribute information of nodes or connections
 - Human subjects can verify whether the extracted communities are coherent
- Evaluation is qualitative
- It is also intuitive and helps understand a community



Evaluation without Ground Truth

- For networks without ground truth or semantic information
- This is the most common situation
- Quantitative evaluation functions
 - Modularity
 - Link prediction (the predicted network is compared with the true network)